

## AI real time chat support system

goal: support users in chat with personalized suggestions,

design; The suggestion should avoid telling user explicitly what to say, except maybe during the first message as an opener, the suggestion itself should also be adjusted to be more and more general and less descriptive to gradually encourage users to do it themselves as the chat goes on, visually, the AI could also get less and less noticeable as the chat gets going, but will also regress to be more helpful if the chat begins to go south again. the character the AI will roleplay as will also differ:

| starting out                                                                                                    | smooth                                                                                   | struggling                                                                                                        | neutral                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| friend of a friend(letting you know about the other person and maybe a few conversation starter, but then dips) | mentor(letting user try themselves while giving more vague hints or even just observing) | adviser(more personalized and helpful advice, should still avoid telling user exactly what to say no matter what) | Facilitator. Provide one nudge to sustain the current light topic, and one optional, low-pressure question to pivot slightly deeper. |

The suggestion generation is done by ai, to balance real-time latency the suggestion generation pipeline is decoupled into two separate processes: an asynchronous background record keeper and a synchronous real-time generator.

note that user profiling is an input, and is read from a json database called appwrite

### Agent 1: background

At the start of every chat, a background agent is initialized. This agent operates silently and does not interact with the user or impact UI latency. it keeps a semantic plan of the chat, while also generating bounds for the generation LLM to follow Trigger: Updates periodically based on message count thresholds or specific heuristic signals (e.g., spikes in response time). Function: It reads the chat log and user profiles to maintain a Semantic Plan (evaluating the chat's health and the AI's current strategic intent). Dynamic Guardrailing: Crucially, this agent is responsible for generating Dynamic Content Boundaries. Because it runs asynchronously without latency constraints, it deeply analyzes the unpredictable human conversation and writes strict, context-specific rules for the real-time generator to follow later note that agent 1's role will not change unlike agent 2 Agent 1 can be openclaw: how openclaw would work:

1. How OpenClaw Reads the Chats (Stateless Execution via Document Fetch)  
OpenClaw remains a stateless function, but the data retrieval is now optimized for a document-based architecture. The Setup: Your primary backend (or an Appwrite

Function) handles the chat events. The Trigger: When the backend detects the threshold (e.g., 5 new messages), it uses the Appwrite SDK to fetch the specific JSON document for that chat session from your SemanticPlans collection. The Execution: The backend sends the user profiles, the new messages, and the fetched JSON document to OpenClaw. OpenClaw runs its "Read-Revise-Rewrite" loop, returns the updated JSON, and the backend instantly updates the Appwrite document using `databases.updateDocument()`. OpenClaw then shuts down.

2. Tracking Plans and Handling Session Decay (Native Metadata) Appwrite makes handling context decay incredibly easy because every document automatically tracks its own metadata. You do not need to manually create a `plan_last_updated` column. The Logic: When triggering the background update, your backend checks the Appwrite document's native `$updatedAt` attribute. Continuous Flow: If `Time.Now - $updatedAt < 12 hours`, the backend passes the existing JSON document to OpenClaw to be revised. Decay Reset: If `Time.Now - $updatedAt > 12 hours`, the context has decayed. The backend bypasses the old document, passes a blank JSON template to OpenClaw, and instructs it: "This is a resumed session after a long break. Treat this as a fresh start and figure out the new vibe." It then overwrites the old Appwrite document with this new baseline.
3. Handling Abandoned Chats (Garbage Collection via CRON) Unlike Redis, NoSQL databases like Appwrite do not have native, automatic "Time-To-Live" (TTL) self-destruct timers for individual documents. To keep your database lean and avoid paying for dead storage, you implement a scheduled cleanup job. The Scheduled Function: You write a lightweight Appwrite Function (a CRON job) set to run automatically once every 24 hours. The Cleanup Sweep: This function queries the SemanticPlans collection for any documents where the `$updatedAt` timestamp is older than 30 days. It iterates through those specific documents and deletes them. Zero manual database maintenance required. The Unexpected Reactivation: If users text each other on Day 31, the backend tries to fetch the document from Appwrite and gets a 404 Document Not Found error. The backend catches this exception and treats it as a brand-new chat, triggering OpenClaw to generate a fresh Semantic Plan JSON from scratch.

SEMANTIC PLAN STRUCTURE:

```
{  
  
  "session_id": "chat_897234",  
  
  "current_role": "Adviser",
```

"chat\_health\_score": 0.8,(0 to 1) //used for transitioning, calculated through signals

"macro\_summary": "Users discussed their college majors. User A is currently venting about a stressful project. User B is giving short, one-word replies.....", //context on the chat

"strategic\_intent": "De-escalate User A's venting and encourage a pivot to User B's interests to balance the conversation.....", //AI's current plan for the chat

"AI\_reliance\_level": 0.4(0 to 1) //used for the AI to see if they should let users try it themselves, used together with chat health score

"user\_A\_id": "u\_5512"

"user\_A\_context": "Stressed about academics. Highly analytical. Needs to be prompted to ask about others." // this would replace the user profiling in agent 2

"user\_B\_id": "u\_9941"

"user\_B\_context": "Passive conversationalist. Enjoys mechanical hobbies. Responds well to direct 'how' questions.",//this would replace the profiling in agent 2

"dynamic\_content\_bounds": [

"STRICT RULE: Do not suggest talking about school, projects, or deadlines.",

"STRICT RULE: Do not let User A ask for advice on their current stress.",

"MUST DO: Suggest a topic related to passive entertainment (movies, music) to lower the tension."

...

]

}

reading and writing:

openclaw would update the current semantic plan to the appwrite DB after the processes above

FALLBACK: if openclaw did not work, we can easily switch to a traditional LLM api, this is also the case for testing

Agent 2:

When the user manually presses the "Help me reply" button, a separate, highly constrained LLM is triggered. This agent does not evaluate the chat's long-term health; it acts strictly as a data-formatting engine.

The Hard Gate (Risk Detection): Before generation, the request passes through a hard gate. If severe risk or toxicity is detected, the AI is bypassed entirely in favor of a hardcoded safety override.

this will change based on how the risk detection works Context Assembly

If safe, the backend retrieves: The latest Semantic Plan and Dynamic Content Boundaries (from Agent 1). A Sliding Window Micro-Context (only the last 3-5 messages) for immediate tactical awareness. suggestion generation To guarantee Agent 2 follows the rules without self-sabotaging or hallucinating, the backend strictly enforces boundaries using API-level role separation:

System Prompt: The backend injects hardcoded Syntax & Formatting Constraints here. This includes the absolute rule to never provide exact dialogue/quotes, and structural laws (e.g., "You are a strict data-formatting engine. Output only in JSON format."). User Prompt: The backend passes the dynamic rules generated by Agent 1 and the sliding window chat log here By passing the constraints via a System Prompt, Agent 2 blindly follows the direct orders of the backend administrator, resulting in a perfectly formatted, contextually safe UI nudge generated fast. Role Transition Matrix & Deterministic State Management

agent 2 can not choose their own roles. The transition between roles (FRIEND, FACILITATOR, MENTOR, ADVISER) is governed strictly by Backend This backend logic calculates raw signals to update the current\_role string within the Semantic Plan database document. Agent 1 updates and Agent 2 simply read this string and act accordingly

The Decision Matrix Signal Ingestion:

Before Agent 1 is triggered, the backend calculates the chat\_health\_score (0.0 to 1.0) based on response latency and heuristics (like message length and mutual reply rates), and the ai\_reliance\_level (0.0 to 1.0) based on the frequency of the user's manual AI calls. (additional signals may be added)

Threshold Evaluation: The backend runs these two floats through a hardcoded if/else matrix to determine the appropriate state. Database Update: The backend overwrites the current\_role field in the active JSON document.

The State Transition Thresholds:

Case 1: FRIEND Condition: message\_count < 5 Action: The system forces high-visibility profile matching to get the strangers talking. (This overrides all health/reliance scores because there isn't enough data yet).

Case 2: ADVISER Condition: chat\_health\_score < 0.3 Action: the user needs help. The AI regresses to maximum visibility and provides concrete rescue strategies to save the interaction.

Case 3: MENTOR Condition: chat\_health\_score > 0.7 Action: The chat is healthy. It doesn't matter if they are dependent or independent; the AI steps back, UI degrades to placeholder text, and it only asks meta-questions.

Case 4: MENTOR Condition: chat\_health\_score is between 0.3 and 0.7 (Implied by surviving Rule 2 and 3) AND ai\_reliance\_level >= 0.6 Action: The chat is stable, but the user is relying on the AI too much. Trigger the fade-out mechanic to force them to think for themselves.

Case 5: FACILITATOR Condition: chat\_health\_score is between 0.3 and 0.7 AND ai\_reliance\_level < 0.6 Action: The chat is stable, and the user is behaving normally. The AI remains visually present but only offers light nudges to sustain the topic.

Visual: based on the roles, the visual will be different:

- High Visibility (Pop-up/Distinct Box): Used for FRIEND and ADVISER.
- Medium Visibility (Standard Chat Button): Used for FACILITATOR.
- Low Visibility (Ghost/Placeholder text in the input bar): Used for MENTOR.

the Ai should be manually triggered

transition between roles should not be too noticeable

after the chat ended(however that will be determined), the AI may ask the users opinion on its performance to better itself